

ChatInputView 实现详解

ChatInputView.swift 实现详解

一、整体功能介绍

ChatInputView.swift 是项目中的聊天交互组件，专注于 “用户与 AI 助手的对话功能”，类似微信 / QQ 聊天界面，支持 **文本消息** 与 **语音消息** 双向交互，核心定位为 “悬浮式快捷聊天入口 + 完整对话界面”，满足用户实时沟通需求。

二、核心组件解析

聊天功能拆分为 6 个核心子组件，各组件职责单一、协同工作，覆盖 “消息模型、入口、录制、对话、显示、动画” 全流程：

1. 基础消息模型（Message）

消息数据的 “最小单元”，定义所有消息属性，为后续组件提供统一数据格式，属性如下：

属性名	类型 / 取值范围	功能描述
id	String (UUID 生成)	消息唯一标识，避免重复
content	String	消息内容（文本消息为文字，语音消息为语音文件路径 / Base64 编码）
isUser	Bool	标记消息发送方（true = 用户发送，false=AI 发送）
timestamp	Date	消息发送时间，用于排序和显示
isThinking	Bool	标记 AI 是否 “正在思考”（- true = 显示 “正在输入” 提示，false = 显示完整回复）

type	MessageType (枚举)	消息类型 (.text = 文本, .taskSuggestion = 任务建议, .taskCreation = 任务创建)
voiceData	Data?	语音消息的二进制数据 (可选, 仅语音消息有值)
voiceDuration	TimeInterval?	语音消息时长 (可选, 单位: 秒, 仅语音消息有值)

补充：MessageType 枚举（消息类型定义）

```
enum MessageType {
  case text          // 纯文本消息
  case taskSuggestion // AI生成的任务建议（点击可创建任务）
  case taskCreation   // 任务创建结果反馈（如“任务已创建成功”）
}
```

2. 主界面（ChatInputView）

聊天功能的“入口组件”，以悬浮按钮形式存在于页面底部，负责触发“完整聊天界面”或“语音录制界面”，核心功能如下：

- **组件形态**：横向排列两个按钮（文本按钮 + 语音按钮），悬浮于页面底部，不遮挡核心内容；
- **交互逻辑**：
 - 点击“文本按钮”：弹出 ChatSheetView（完整聊天界面），支持文本输入与历史对话查看；
 - 点击“语音按钮”：弹出 VoiceRecordingView（语音录制界面），支持语音消息录制；
- **消息同步**：监听系统通知（如 Notification.Name("NewMessageReceived")），收到新消息时更新入口状态（如显示未读提示）。

3. 语音录制界面（VoiceRecordingView）

专门用于“语音消息录制”的弹窗组件，支持“录制、暂停、取消、提交”全流程，核心功能如下：

- **界面元素**：
 - 动态波形动画：实时显示录音音量波动，反馈录制状态；

- 时长显示：实时更新录音时长（如 “00:05” ），限制最大录制时长（如 60 秒）；
- 操作按钮：暂停 / 继续按钮（录制中显示 “暂停” ，暂停时显示 “继续” ）、取消按钮（放弃录制）、完成按钮（提交语音消息）；
- 核心逻辑：
 - 录制开始：调用系统麦克风权限，启动音频录制；
 - 暂停 / 继续：暂停时保留已录制数据，继续时追加录制；
 - 提交完成：将语音数据转换为 Message 模型，调用 MessageService 保存，同时跳转到 ChatSheetView 显示新消息。

4. 完整聊天界面（ChatSheetView）

用户与 AI 对话的 “核心交互界面” ，类似传统聊天 App 的对话页，核心功能如下：

- 界面结构：
 - 顶部导航栏：显示 “AI 助手” 标题，右侧提供 “清除所有消息” 按钮；
 - 中间消息列表：垂直排列所有历史消息（MessageRow 组件），自动按时间倒序排序；
 - 底部输入区：文本输入框 + 语音录制按钮，支持输入文本或触发语音录制；
- 关键交互：
 - 自动滚动：新消息发送 / 接收时，列表自动滚动到最新消息；
 - 文本提交：输入框回车或点击 “发送” 按钮，提交文本消息；
 - 语音触发：点击输入区 “语音按钮” ，跳转至 VoiceRecordingView；
 - 清除消息：点击导航栏 “清除” 按钮，调用 MessageService 删除所有本地消息，同时刷新列表。

5. 单条消息显示（MessageRow）

“消息列表的子组件” ，负责渲染单条消息的样式，根据 “发送方、消息类型” 显示不同 UI，核心功能如下：

- 布局区分：
 - 用户发送的消息（isUser=true）：右对齐，背景为主题色（如紫色），文字为白色；
 - AI 发送的消息（isUser=false）：左对齐，背景为浅灰色，文字为黑色；
- 类型适配：

- 文本消息：直接显示 `content` 内容，AI 消息支持 `TypewriterText`（打字机动画）；
- 语音消息：显示 “语音波形图标 + 时长”，点击可播放语音，播放时波形动画同步动效；
- 任务建议消息（`.taskSuggestion`）：文本下方附加 “创建任务” 按钮，点击触发任务创建逻辑；
- **附加信息**：每条消息下方显示发送时间（如 “14:30” ），AI “正在思考” 时显示 “正在输入...” 提示。

6. 打字机效果（TypewriterText）

增强 AI 消息交互感的 “动画组件”，让文字以 “逐个字符弹出” 的方式显示，模拟 “AI 实时输入” 的效果，核心逻辑如下：

- **实现原理**：通过 `Timer` 定时从完整文本中截取 “前 n 个字符”，更新显示内容，直至显示全部文本；
- **可配置项**：
 - 打字速度：默认每 0.1 秒显示一个字符，可通过参数调整；
 - 完成回调：动画结束后触发回调（如更新消息状态为 “已完成” ）；
- **使用场景**：仅用于 AI 发送的文本消息，提升用户感知（避免一次性显示大段文本的生硬感）。

三、数据获取逻辑

聊天数据的 “加载、传递、更新” 遵循 “本地存储优先、多组件协同” 原则，确保数据一致性与可用性：

1. 数据来源

聊天数据主要来自 3 个渠道，覆盖 “历史记录、用户输入、AI 反馈”：

数据来源	获取方式	数据类型
本地存储	通过 <code>MessageService</code> 的 <code>loadMessages()</code> 方法，从本地（如 <code>UserDefaults</code> 、 <code>CoreData</code> ）加载历史消息	[<code>Message</code>]（消息数组）
用户输入	文本输入框获取文字，语音录制获取音频数据，转换为 <code>Message</code> 模型	<code>Message</code> （单条用户消息）

AI 回复

通过 ChatController（聊天控制器）调用 API 获取 AI 回复，转换为 Message 模型

Message（单条 AI 消息）

补充：MessageService 核心方法（本地存储管理）

```
class MessageService {  
    // 单例模式，全局统一管理  
    static let shared = MessageService()  
  
    // 加载本地历史消息  
    func loadMessages() -> [Message] {  
        guard let data = UserDefaults.standard.data(forKey: "chat_messages"),  
              let messages = try? JSONDecoder().decode([Message].self, from: data) else {  
            return [] // 无本地数据时返回空数组  
        }  
        return messages.sorted(by: { $0.timestamp < $1.timestamp }) // 按时间升序排序  
    }  
  
    // 保存消息到本地  
    func saveMessage(_ message: Message) {  
        var messages = loadMessages()  
        messages.append(message)  
        if let data = try? JSONEncoder().encode(messages) {  
            UserDefaults.standard.set(data, forKey: "chat_messages")  
        }  
    }  
  
    // 清除所有本地消息  
    func clearAllMessages() {  
        UserDefaults.standard.removeObject(forKey: "chat_messages")  
    }  
}
```

2. 数据传递方式

采用“多方式协同”的数据传递策略，适配不同组件间的交互场景：

传递方式	适用场景	实现方式
绑定（Binding）	父子组件间实时同步数据（如 ChatSheetView 与输入框的文本同步）	父组件通过 @State 定义数据，子组件通过 @Binding 接收，数据变化双向同步
控制器共享	跨组件共享消息状态（如 ChatInputView 与 ChatSheetView 共享消息列表）	定义 ChatInputController（ObservableObject），组件通过 @ObservedObject 引用，数据变化自动通知
通知机制	无直接关联的组件间传递消息更新（如 AI 回复到达后通知所有相关组件）	通过 NotificationCenter- 发送 / 接收通知（如 “NewMessageReceived”），触发组件刷新

补充：ChatInputController 核心逻辑（共享控制器）

```
class ChatInputController: ObservableObject {
    @Published var messages: [Message] = [] // 所有消息，@Published确保变化时通知视图
    private let messageService = MessageService.shared

    // 初始化：加载本地消息
    init() {
        messages = messageService.loadMessages()
    }

    // 添加新消息（自动保存到本地）
    func addMessage(_ message: Message) {
        messages.append(message)
        messageService.saveMessage(message)
        // 发送通知，告知其他组件“有新消息”
        NotificationCenter.default.post(name: NSNotification.Name("NewMessageReceived"),
        object: nil)
    }

    // 清除所有消息
    func clearMessages() {
        messages.removeAll()
    }
}
```

```
messageService.clearAllMessages()
    }
}
```

3. 数据加载流程

从“界面初始化”到“消息更新”的完整数据流转，确保组件启动时加载历史、交互时同步新数据：

1. 步骤 1：界面初始化加载历史消息

- ChatInputView/ChatSheetView 初始化时，通过 ChatInputController 的 init() 方法，调用 MessageService.loadMessages() 加载本地历史消息；
- 加载的消息数组同步到 ChatInputController 的 messages 属性，组件通过 @ObservedObject-监听，自动渲染消息列表。

2. 步骤 2：同步消息到控制器与界面

- 新消息产生（用户输入 / AI 回复）时，调用 ChatInputController.addMessage(_) 方法，将新消息添加到 messages 数组；
- 同时调用 MessageService.saveMessage(_) 保存到本地，确保下次启动时可恢复；
- 发送 “NewMessageReceived” 通知，所有监听该通知的组件（如 ChatInputView）收到后执行刷新逻辑（如更新未读提示）。

3. 步骤 3：收到新消息后刷新界面

- 组件（如 ChatSheetView）通过 onReceive 监听 “NewMessageReceived” 通知，收到后触发列表自动滚动到最新消息；
- 若消息为 AI “正在思考” 状态 (isThinking=true)，MessageRow 显示 “正在输入” 提示；AI 回复完成后，更新消息 content 与 isThinking，界面自动刷新为完整回复。

（注：文档部分内容可能由 AI 生成）